Project - Low Level Design

On

KnowDocs (RAG)

Datagami GEN AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Sneha Jain	EN22CS301966
2	Soumya Barve	EN22CS301975
3	Sumit Pawar	EN22CS3011000
4	Vikas Devnani	EN22CS3011080
5	Vikas Dhakad	EN22CS3011081

Group Name: D909

Project Number: G-09

Industry Mentor Name:

University Mentor Name: Prof. VINEETA RATHORE

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.
2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.
3. Diagrams
4. References

1. Introduction

1.1. Scope of the Document

This document defines the low-level architecture and implementation details for a document-based question-answering system. It details the internal logic of the Python modules, data transformation steps, and the integration of local embedding models with high-speed inference APIs.

1.2. Intended Audience

The primary audience includes software engineers, AI developers, and system architects responsible for the deployment, maintenance, or further scaling of the NLP pipeline.

1.3. System Overview

The system is a Retrieval-Augmented Generation (RAG) platform that allows users to interact with technical PDF documents. By utilizing semantic similarity search and grounded prompt engineering, the system provides precise answers while strictly preventing AI hallucinations.

2. System Design

2.1. Application Design

The application follows a modular "Free Stack" design. It is built using **Python 3.13** and **LangChain** for orchestration. The user interface is handled by **Streamlit**, while high-performance inference is offloaded to **Groq's LPU** hardware.

2.2. Process Flow

The system executes two primary workflows:

- **Ingestion Flow:** PDF Upload → Text Cleaning → Chunking → Local Embedding → FAISS Indexing.
- **Query Flow:** User Question → Query Embedding → Similarity Search → Contextual Prompting → LLM Generation.

2.3. Information Flow

1. **Extraction:** Text is pulled from PDFs using a primary (pypdf) and fallback (pdfminer.six) strategy.

2. **Transformation:** Text is split into chunks of **500 characters** with a **50-character overlap** to maintain semantic continuity.
3. **Vectorization:** Chunks are converted into **384-dimensional** dense vectors using the all-MiniLM-L6-v2 model.
4. **Retrieval:** The system searches the **FAISS** index for the **top 4** most relevant context blocks based on L2 distance.

2.4. Components Design

Component	Module	Responsibility
Ingestion Manager	pdf_loader.py	Handles multi-file uploads and raw byte extraction.
Text Splitter	text_splitter.py	Implements recursive character splitting to preserve paragraphs.
Vector Engine	embeddings.py	Manages the local HuggingFace embedding model.
Vector Store	faiss_store.py	Performs L2 similarity search and manages local index persistence.
Generation Engine	groq_llm.py	Orchestrates LLM calls to Llama-3.3 using grounded prompts.

2.5. Key Design Considerations

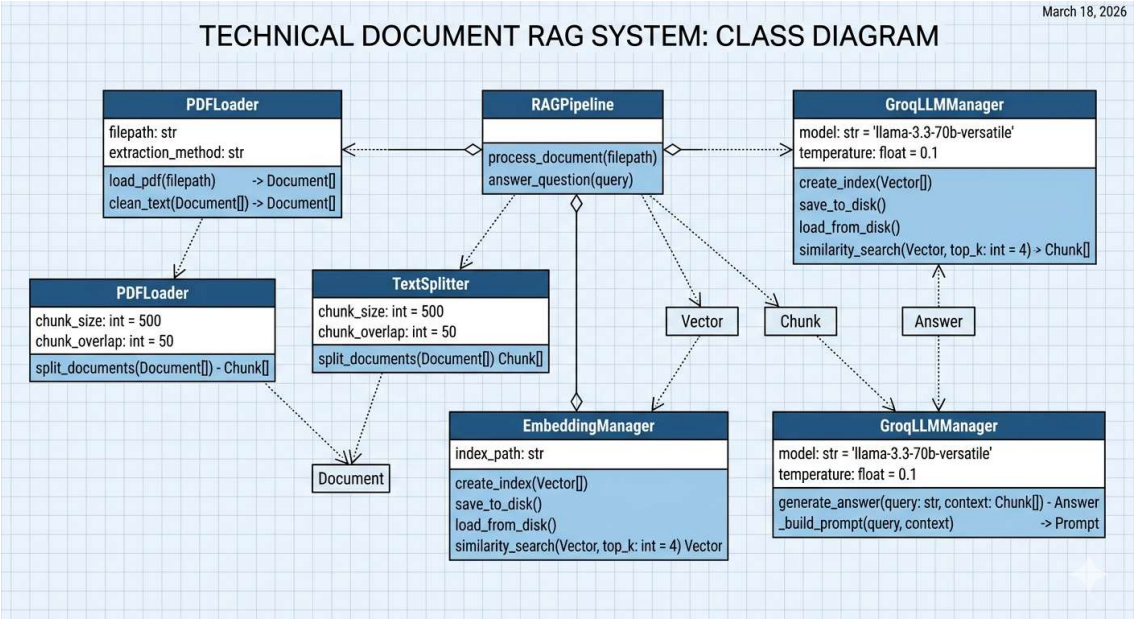
- **Zero Hallucination:** The system is designed with a 4-layer prevention strategy, including strict prompt rules and a low temperature of **0.1** for deterministic responses.
- **Resource Efficiency:** Uses a small **22MB** local embedding model that runs entirely on **CPU**, eliminating the need for expensive GPUs.
- **Persistence:** The FAISS index is stored locally as binary (.faiss) and pickle (.pkl) files to allow instant recovery between sessions.

2.6. API Catalogue

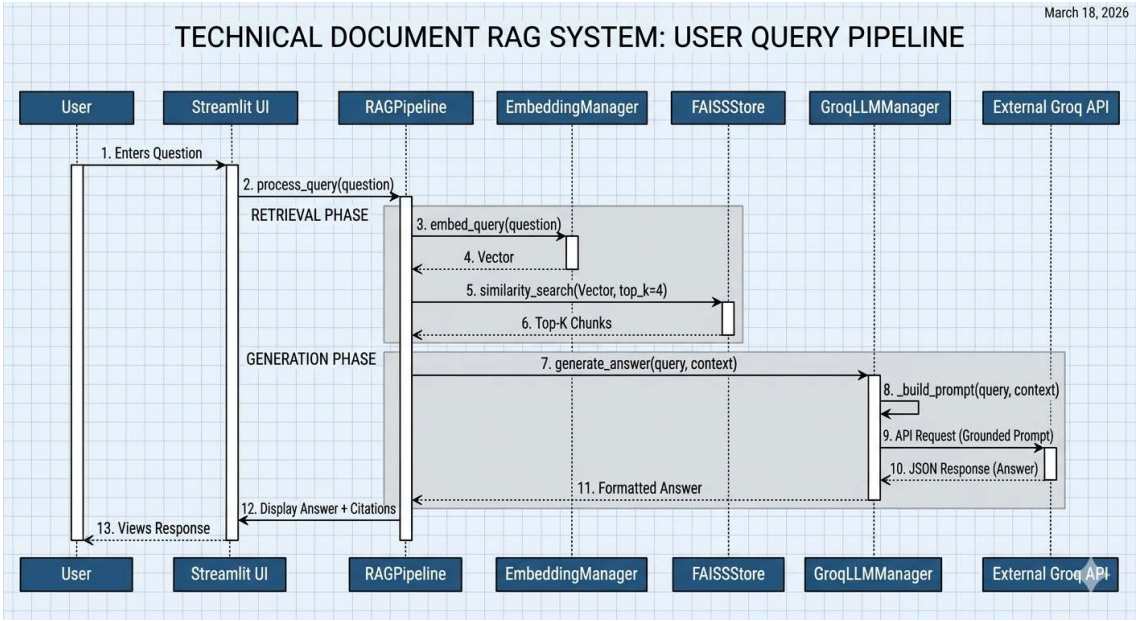
The system primarily interfaces with the **Groq API**:

- **Base URL:** https://api.groq.com/openai/v1.
- **Primary Model:** llama-3.3-70b-versatile.
- **Security:** Authorization via Bearer Token.

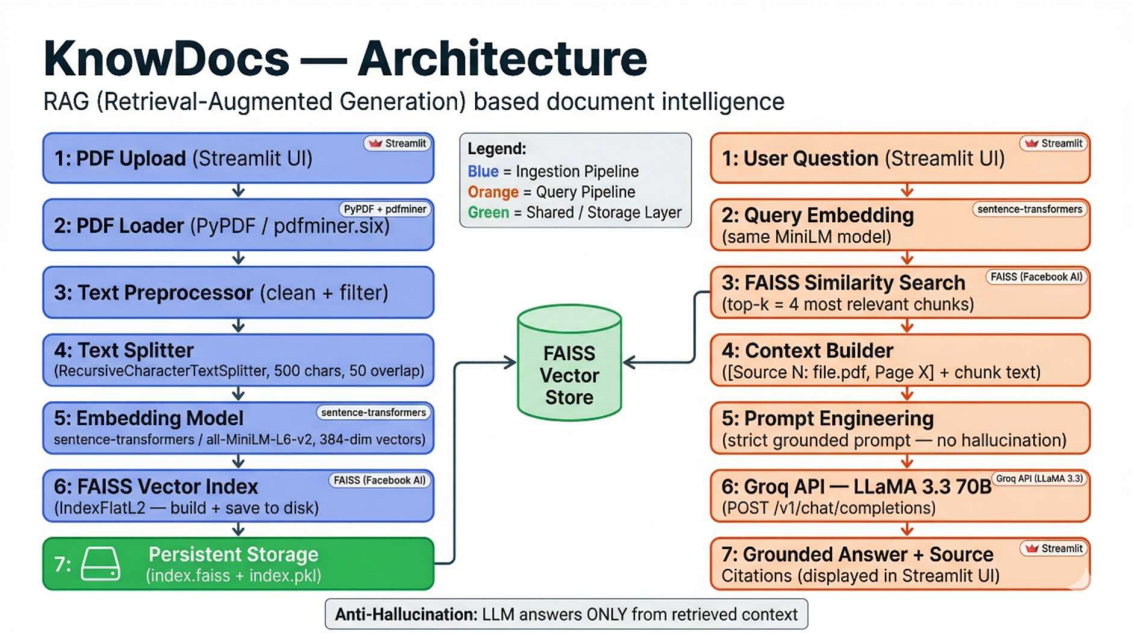
3. DIAGRAMS



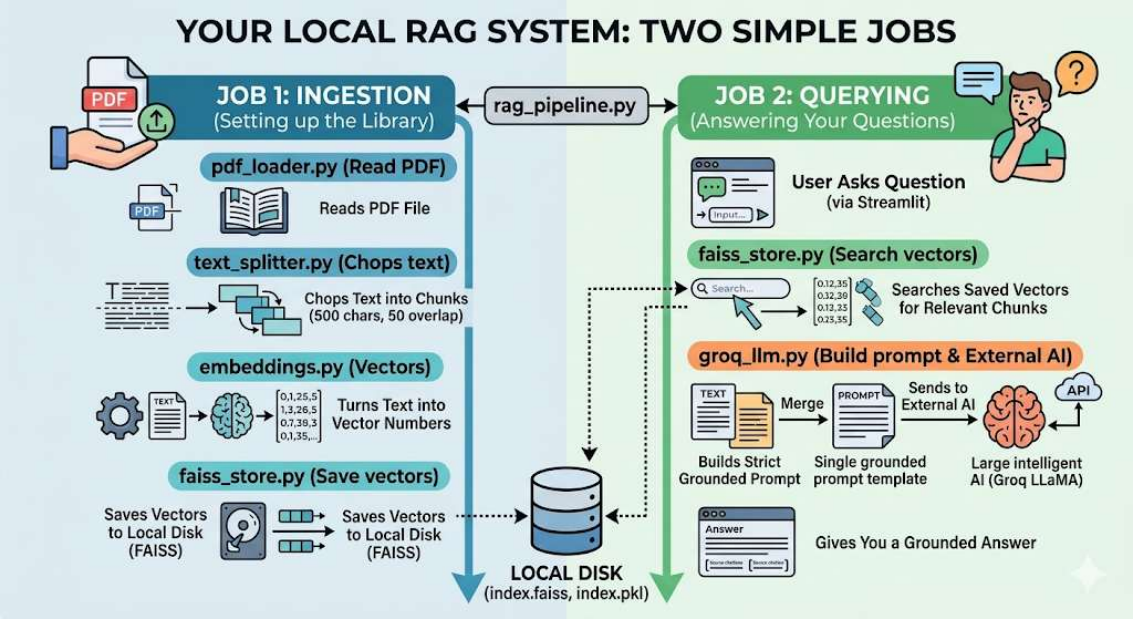
3.1 CLASS DIAGRAM



3.2 SEQUENCE DIAGRAM



3.3 ARCHITECTURE DIAGRAM



3.4 API CATLOG

4. References

- Python 3.13: Core language.
- LangChain (0.2.16+): RAG orchestration framework.
- FAISS-CPU (1.13.2+): Vector similarity search library.
- Sentence-Transformers (5.3.0+): Local embedding generation.
- Streamlit (1.55+): Web UI development.